Managing Microsoft Purview Going Forward — Operations & Governance

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Managing Microsoft Purview Going Forward: Operations & Governance

The situation . . . 5

The solution . . . 5

Why it matters . . . 5

What stays with your governance functions . . . 6

Bottom line . . . 6

1. Solution Overview — Managing Microsoft Purview Going Forward: Operations & Governance 6

What this solution is . . . 6

Why this matters . . . 6

How it works . . . 7

The operating cadences (the heartbeat of the program) . . . 8

What you can control . . . 8

What stays with your governance functions (be explicit with buyers) . . . 8

Licensing reality (verify for your tenant) . . . 8

The standard operating model . . . 9

2. Requirements Guide — Managing Microsoft Purview: Operations & Governance 9

2.1 An implemented Purview foundation to operate . . . 9

2.2 Purview operations roles (least privilege, role-separated) . . . 9

2.3 S&C; PowerShell + Graph, plus Graph Security permissions for alerting . . . 10

2.4 A recorded policy baseline (for drift detection) . . . 10

2.5 Compliance Manager access and an improvement-action export . . . 10

2.6 An approved role-membership list (access-review baseline) . . . 10

2.7 Defined alert-triage scope, severities, and SLAs . . . 11

2.8 A defined KPI set, thresholds, and review governance . . . 11

2.9 Change-control process and window for live-policy edits . . . 11

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 12

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

3. Operating Guide — Managing Microsoft Purview: Operations & Governance 14

Named operating artifacts . . . 14
Part A — Stand up the operating model (once) 14
Step 1 — Baseline the estate (Assess) . . . 15
Step 2 — Define the policy lifecycle + change control (Configure) . . . 15
Step 3 — Wire alert triage + reporting (Configure) . . . 15
Step 4 — Set up the Compliance Manager cadence (Configure) . . . 15
Step 5 — Schedule the role & access review (Configure) . . . 16
Step 6 — Stand up the KPI dashboard + review calendar (Configure) . . . 16
Part B — Run the program (recurring heartbeat) 16
RB-01 — Policy lifecycle & change control (weekly drift + quarterly review) . . . 16
RB-02 — Compliance Manager improvement cadence (monthly) . . . 17
RB-03 — Alert triage & incident rollup (daily triage, weekly rollup) . . . 17
RB-04 — Role & access review (quarterly) . . . 17
RB-05 — KPI dashboard & program-health review (monthly refresh, quarterly/annual review) . . . 17
5. Testing & Validation Guide — Managing Microsoft Purview: Operations & Governance 18
Test 1 — Policy inventory + drift detection (RB-01) . . . 18
Test 2 — Config helper is safe by default (RB-01 / change control) . . . 18
Test 3 — Alert rollup across categories (RB-03) . . . 19
Test 4 — Role & access review diff (RB-04) . . . 19
Test 5 — Compliance Manager cadence + KPI dashboard (RB-02 / RB-05) . . . 19
Test 6 — The cadence actually fires (operating model) . . . 20
Acceptance gate (all must pass before declaring the program "operating") . . . 20
6. Troubleshooting Guide — Managing Microsoft Purview: Operations & Governance 20
Connection & permissions . . . 20
RB-01 — Policy lifecycle & drift . . . 21
RB-03 — Alert triage & rollup . . . 22
RB-04 — Role & access review . . . 22
RB-02 / RB-05 — Compliance Manager & KPIs . . . 22
General . . . 23
7. Best Practices — Managing Microsoft Purview: Operations & Governance 23
Policy lifecycle & change control . . . 24
Alert triage & reporting . . . 24
Compliance Manager cadence . . . 24
Role & access reviews . . . 25
KPIs & program health . . . 25
Automation & safety . . . 25

Operating discipline (the short version) . . . 26
8. Deliverables Summary — Managing Microsoft Purview: Operations & Governance 26
Documentation . . . 26
PowerShell tools (5) — Security & Compliance PowerShell / Microsoft Graph . . . 27
Named operating artifacts (created and reused verbatim) . . . 28
Operating cadences . . . 28
Outcome . . . 28
Boundaries (what this solution does not decide) . . . 28
Appendix A — PowerShell Scripts 30

Executive Summary — Managing Microsoft Purview Going Forward: Operations & Governance

The situation

Deploying Microsoft Purview is a project; **running** it is a program. After go-live, the risk shifts from "is it configured?" to "is it *still* working?" Policies drift as the business changes, alerts pile up unless someone triages them daily, Compliance Manager improvement actions lapse and the compliance score decays, role memberships accrete into stale privilege, and the dashboards that once proved health go stale. Left unmanaged, a Purview investment quietly degrades — and the gap is only discovered during an audit or an incident.

The solution

A complete **operating model** for a Purview program that is already implemented — the answer to *how do we run this going forward?* It provides:

- A **policy lifecycle** — design → **simulate/test** → enforce → review — so every change is piloted before

it enforces, reviewed on a cadence, and reversible.

- A **Compliance Manager** improvement-action cadence with owners, due dates, and a monthly burn-down.
- **Alert triage and reporting** across DLP, sensitivity labels, insider risk, and audit, with severities,

SLAs, and a periodic rollup that drives tuning.

- Periodic **role and access reviews** that attest Purview privilege against an approved baseline.
- Lightweight **change control** so live-policy edits are reviewed, timed, recorded, and reversible.
- **KPIs and dashboards** — policies enforced vs test, alert volume and triage time, improvement-action

burn-down, compliance-score trend, privileged-role count — that turn "is the program healthy?" from an opinion into evidence.

Five **read-only-by-default** PowerShell tools (Security & Compliance PowerShell / Microsoft Graph) drive the work: policy inventory + drift detection, alert rollup, role/access review, a Compliance Manager KPI report, and a change-controlled config helper that makes no change without an explicit `-Commit` and a change record.

Why it matters

- **Effective:** policies stay tuned to the business; alerts get worked; improvement actions get done.
- **Tuned:** a falling false-positive rate and a rising compliance score, measured, not assumed.
- **Defensible:** on a cadence, with attested roles, recorded changes, and KPIs — exactly what an audit

wants to see.

- **Safe:** high-impact changes are always piloted in simulation/test first, and every change is reversible

and logged.

What stays with your governance functions

This solution delivers the **technical operating method and the evidence** — not the compliance decisions. Retention schedules, records file plans, legal holds, and insider-risk / communication-compliance case outcomes remain with your compliance, records, privacy, HR, and legal teams, with strict role separation and privacy-by-design for HR-sensitive capabilities. Purview is heavily **licensing-gated** and the portal is evolving — capability availability is verified for your tenant, never assumed.

Bottom line

Microsoft Purview run as a **living program** — reviewed, triaged, improved, attested, and measured — stays effective and defensible year over year, at a fraction of the cost of ongoing external consulting, and with the discipline that keeps a compliance investment from silently decaying.

1. Solution Overview — Managing Microsoft Purview Going Forward: Operations & Governance

What this solution is

Turning Microsoft Purview *on* — labels, DLP, retention, insider risk, audit — is a project with an end date. **Running** Purview is not: policies drift as the business changes, alerts pile up if nobody triages them, Compliance Manager improvement actions stall, role memberships accrete, and the dashboards that once proved the program was healthy go stale. This solution is the **operating model** for a Purview program that is already implemented — a repeatable way to *manage it going forward* so it stays effective, tuned, and defensible year over year. It provides a **policy lifecycle** (design → simulate → enforce → review), a **Compliance Manager** improvement-action cadence, **alert triage and reporting** across DLP / sensitivity labels / insider risk / audit, periodic **role and access reviews**, a light-weight **change-control** discipline, and the **KPIs and dashboards** that demonstrate program health — all driven from the Microsoft Purview portal, Security & Compliance PowerShell (`Connect-IPPSSession`), and Microsoft Graph, with no third-party tooling.

This is deliberately **distinct from an implementation project**. It assumes the foundation exists (the separate *Implementing Microsoft Purview — Foundation, Roles & Onboarding* solution builds that) and answers the question every compliance owner faces after go-live: *how do we operate this?*

Why this matters

- **Policies silently drift.** A DLP or auto-labeling policy left in **simulation/test** mode never enforces; one flipped to enforce without review can block business. Without a lifecycle and a review cadence, nobody can say *which* policies are actually protecting data today.
- **Alerts are worthless if not triaged.** DLP, insider-risk, and audit signals generate a stream of alerts; an unowned queue means real incidents sit unseen and false positives never get tuned out.

- **Compliance Manager score decays.** Improvement actions have owners and due dates; without a monthly

cadence they lapse, your compliance score drops, and audits find gaps that were "assigned" but never done.

- **Roles accrete.** People change jobs, projects end, but Compliance Administrator / Insider Risk /

eDiscovery memberships stay. Stale privilege is an audit finding and a real risk — periodic access review is the control.

- **Change without control is how you break production.** A well-meaning edit to a live retention or DLP

policy, made off-hours with no record, is the classic Purview outage. Lightweight change control makes every change reviewed, reversible, and logged.

- **If you can't measure it, you can't defend it.** Regulators and leadership ask "is the program

working?" KPIs — policies in enforce vs test, alert volume and triage time, improvement-action burn-down, privileged-role count — turn that from an opinion into evidence.

How it works

1. **Baseline the estate.** `PURVOPS-Discovery.ps1` inventories every DLP, sensitivity-label,

auto-labeling, and retention policy with its current **mode** (Enable / Test / Disable) and compares it to a saved baseline to surface **drift** — new, removed, or mode-changed policies since the last review.

1. **Run the policy lifecycle.** Every policy moves through **design** → **simulate/test** → **enforce** → **review**

on a schedule. New and changed policies are always piloted in **simulation/test** first; enforcement is a reviewed, recorded promotion; each live policy is re-reviewed on a cadence and can be rolled back.

1. **Work the Compliance Manager cadence.** Improvement actions are pulled (exported from the Compliance

Manager portal — no public API), tracked against owners and due dates, and burned down monthly, with the compliance score trend recorded as a KPI.

1. **Triage alerts and roll them up.** A daily triage owner works the alert queue across DLP, labels,

insider risk, and audit; `PURVOPS-Report.ps1` produces a periodic **rollup** — volume by category, severity, and status — so tuning and reporting are data-driven.

1. **Review roles and access.** On a quarterly cadence, `PURVOPS-Validation.ps1` enumerates every Purview

compliance role group and its members and compares them to an approved list, flagging additions, departures, and privileged accounts for attestation.

1. **Measure and prove health.** `PURVOPS-KpiReport.ps1` composes the **KPI dashboard** from live

telemetry (policy-mode distribution, alert volume, label/DLP coverage) plus the Compliance Manager export, and a periodic **program-health review** walks the dashboard with stakeholders.

1. **Control every change.** `PURVOPS-Remediation.ps1` performs operational config changes (e.g. promoting

a policy from test to enforce, or rolling it back) `-WhatIf` **by default**, only writing with an explicit `-Commit`, and stamps each change into the change record.

The operating cadences (the heartbeat of the program)

Cadence	Activity	Runbook
Daily	Alert triage — work the DLP / insider-risk / audit alert queue	RB-03
Weekly	Policy-drift check + alert rollup review	RB-01, RB-03
Monthly	Compliance Manager improvement-action burn-down; KPI dashboard refresh	RB-02, RB-05
Quarterly	Role & access review; live-policy review pass	RB-04, RB-01
Annually	Full program-health review; scope/coverage reassessment	RB-05

What you can control

- Which policy families are in the review scope (DLP, sensitivity/auto-labeling, retention) and the review interval.
- The alert categories, severities, and SLAs that define the triage queue and its response times.
- The Compliance Manager improvement-action owners, due-date policy, and score target.
- The approved Purview role membership and the access-review interval (e.g. quarterly).
- The KPI set and thresholds that define "healthy," and who attends the program-health review.
- The change-control approval path and change window for live-policy edits.

What stays with your governance functions (be explicit with buyers)

This solution delivers the **technical operating method and the evidence** — not the compliance *determinations*. **Retention schedules, records file plans, legal holds, and insider-risk / communication-compliance case actions and outcomes** remain with your compliance, records, privacy, HR, and legal teams. Insider-risk and communication-compliance operations are HR/privacy-sensitive: this model emphasizes **privacy-by-design, pseudonymization where the feature supports it, and strict role separation** between who investigates and who administers. We give you the runbooks, cadences, reviews, and dashboards; the regulatory and case decisions stay with the accountable owners.

Licensing reality (verify for your tenant)

Purview capabilities are heavily **licensing- and plan-dependent** and the portal is **actively evolving**. Advanced DLP, auto-labeling simulation, insider risk, communication compliance, Audit (Premium), and DSPM generally require **Microsoft 365 E5** or the **E5 Compliance** add-on; some features are in preview. Compliance Manager improvement-action detail and premium templates are also plan-gated, and Compliance Manager has **no supported public API** — its data is exported from the portal. Every step below is flagged where availability depends on licensing; **confirm capability and portal paths for your own tenant** before relying on them.

The standard operating model

> *Baseline the estate → run the policy lifecycle (design → **simulate/test** → enforce → review) → work the Compliance Manager cadence → triage alerts and roll them up → review roles and access → measure and prove health → control every change. High-impact policy changes are **always piloted in simulation/test before enforcement**, and every change is reversible and recorded.*

2. Requirements Guide — Managing Microsoft Purview: Operations & Governance

Verify **every** requirement before you begin operating. Each entry gives **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant licensing, Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, governance stakeholders, and a pilot scope + change control) are in `global-requirements-purview.md` — this file adds the prerequisites specific to **running Purview as an operated program**. Reference the six category requirements; do not re-derive them.

> ***This is an operations solution, not an implementation.*** *It assumes an **already-implemented Purview foundation** (labels, DLP, retention, and the relevant advanced capabilities are live). If that foundation does not yet exist, implement it first (see the separate *Implementing Microsoft Purview — Foundation, Roles & Onboarding* solution) — you cannot operate what has not been built.*

2.1 An implemented Purview foundation to operate

Description: Live Purview configuration to manage — at minimum published sensitivity labels and label policies, DLP policies, and retention policies; optionally auto-labeling, insider risk, communication compliance, and Audit (Premium) if in scope. **Why:** The entire operating model (lifecycle, drift, triage, KPIs) acts on existing policies; with nothing deployed there is nothing to operate. **Verify:** In the Purview portal you can see published policies under **Data loss prevention** → **Policies**, **Information protection** → **Sensitivity labels / Label policies**, and **Data lifecycle management** / **Records management** → **Retention policies**. `PURVOPS-Discovery.ps1` returns a non-empty inventory. **Implement:** Complete the implementation project first, then return to operate it.

2.2 Purview operations roles (least privilege, role-separated)

Description: The **least-privilege** compliance role groups the operating tasks need — typically **Compliance Administrator** (or **Compliance Data Administrator**) for policy lifecycle and change, **Information Protection Admin/Analyst** for labels/DLP, **Insider Risk Management** roles for insider-risk operations, **eDiscovery** and **Audit** roles where those are in scope — with a deliberate **separation** between who *administers* and who *investigates* (especially for insider risk and communication

compliance). Global Administrator is used only where genuinely required. **Why:** Purview uses fine-grained role groups; operating a capability requires the matching role, and HR/privacy-sensitive capabilities require role separation as a control, not a convenience. **Verify:** Purview portal → **Roles & scopes** → **Permissions** (or **Settings** → **Roles**) shows your accounts in the named role groups. `PURVOPS-Validation.ps1` enumerates them read-only. **Implement:** Assign the least-privilege role groups under change control; document the separation between administrator and investigator/reviewer duties for insider risk and communication compliance.

2.3 S&C PowerShell + Graph, plus Graph Security permissions for alerting

Description: A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via `ExchangeOnlineManagement`) and **Microsoft Graph PowerShell**; for the alert rollup, the Graph **security** read scopes (`SecurityEvents.Read.All` or `SecurityAlert.Read.All`) consented. **Why:** Policy inventory, role review, and config changes run through S&C PowerShell; the alert rollup reads triggered alerts from the Microsoft Graph security API. **Verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account; a read against `/security/alerts_v2` returns without a permissions error. Run `PURVOPS-Report.ps1` — it reports if a scope is missing. **Implement:** `Install-Module ExchangeOnlineManagement` and `Install-Module Microsoft.Graph`; consent the security read scopes at first `Connect-MgGraph` (admin consent once if required).

2.4 A recorded policy baseline (for drift detection)

Description: A saved snapshot of the current policy inventory (names + modes) that "healthy" is measured against — the baseline `PURVOPS-Discovery.ps1` compares each run to. **Why:** Drift is only meaningful relative to a known-good baseline; without one you can list policies but not detect *what changed* since the last review. **Verify:** A baseline JSON exists (`Policy Baseline – Purview – <date>`); a fresh discovery run diffs cleanly against it. **Implement:** Run `PURVOPS-Discovery.ps1 -SaveBaseline` once after a reviewed, known-good state to establish the baseline; re-baseline after each approved change set.

2.5 Compliance Manager access and an improvement-action export

Description: Access to **Compliance Manager** in the Purview portal and the ability to **export** its improvement actions (Compliance Manager has **no supported public API** — data is exported from the portal as CSV). **Why:** The improvement-action cadence and the compliance-score KPI are driven from Compliance Manager; the KPI script ingests the exported CSV. **Verify:** Purview portal → **Compliance Manager** → **Improvement actions** loads; the **Export** control produces a CSV. `PURVOPS-KpiReport.ps1 -ComplianceManagerCsv <path>` ingests it. **Implement:** Confirm licensing exposes the improvement actions you need (some templates/actions are premium/E5), and establish the monthly export as part of the cadence.

2.6 An approved role-membership list (access-review baseline)

Description: An **authoritative list of who should be in each Purview role group** — the attestation baseline the quarterly access review compares actual membership to. **Why:** An access review needs an approved state to compare against; otherwise you can list members but not identify unauthorized or stale privilege. **Verify:** An approved-membership CSV exists and is owned by a named approver; `PURVOPS-Validation.ps1 -ApprovedListCsv <path>` reports additions/removals against it.

Implement: Capture current approved membership with the accountable owner, store it as the baseline, and re-approve it each review cycle.

2.7 Defined alert-triage scope, severities, and SLAs

Description: The **alert categories in scope** (DLP, insider risk, information-protection/label, audit), their **severities**, and the **response SLAs** (e.g. high-severity triaged within X hours) that define the triage queue. **Why:** "Triage the alerts" is only operable when the queue, priorities, and response times are defined; SLAs make triage measurable as a KPI. **Verify:** The categories, severities, and SLAs are documented and agreed; the alert rollup groups by those categories. **Implement:** Agree the scope and SLAs with security/compliance stakeholders and record them in the operating runbook before triage begins.

2.8 A defined KPI set, thresholds, and review governance

Description: The **KPIs** that define program health (e.g. % policies enforced vs test, alert volume and mean triage time, improvement-action burn-down, compliance-score trend, privileged-role count), their **thresholds**, and the **governance forum** (who reviews, how often). **Why:** Proving health requires an agreed measure and a forum that acts on it; undefined KPIs produce a dashboard nobody owns. **Verify:** The KPI list with thresholds is documented; a recurring program-health review is scheduled with named attendees. **Implement:** Agree the KPI set and thresholds with leadership/compliance, schedule the review cadence, and set the dashboard up to report exactly those measures.

2.9 Change-control process and window for live-policy edits

Description: A **change-control** path — request, review/approval, change window, recorded change, rollback plan — for edits to live DLP, label, and retention policies. **Why:** Editing a live, enforcing policy is high-impact; uncontrolled changes are the classic Purview outage. Change control makes every change reviewed, timed, recorded, and reversible. **Verify:** A change-record store exists (`Change Record – Purview – ChangeControl`) and a change window / approver are named; `PURVOPS-Remediation.ps1` defaults to `-WhatIf` and stamps a change record on `-Commit`. **Implement:** Define the approval path and window with stakeholders; adopt the `-WhatIf`-first discipline and the change record for every operational change.

Checklist

- Implemented Purview foundation present to operate (2.1)
- Least-privilege, role-separated operations roles assigned (2.2)
- S&C PowerShell + Graph installed; Graph security read scopes consented (2.3)
- Policy baseline recorded for drift detection (2.4)
- Compliance Manager export established (no public API) (2.5)
- Approved role-membership list captured as the access-review baseline (2.6)
- Alert-triage scope, severities, and SLAs defined (2.7)
- KPI set, thresholds, and program-health review forum agreed (2.8)

• Change-control path, window, and change record in place; `-WhatIf`-first adopted (2.9)

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Operating Guide — Managing Microsoft Purview: Operations & Governance

This is the **operating** guide: how to stand up and then *run* the Purview program on a cadence. Work through **Part A** once to establish the operating model, then run **Part B** as the recurring heartbeat. Inventory, roles, and config come from **Security & Compliance PowerShell** (`Connect-IPPSSession`) and the Microsoft Purview portal; triggered alerts come from **Microsoft Graph** (`/security/alerts_v2`). Every config change is run `-WhatIf` **first** and only committed with `-Commit` under change control. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention and are reused verbatim across docs, scripts, and the portal.

> ***Model:** Baseline → run the policy lifecycle (design → **simulate/test** → enforce → review) → work the Compliance Manager cadence → triage + roll up alerts → review roles → measure health → control every change. High-impact policy changes are **always piloted in simulation/test before enforcement**.*

Named operating artifacts

Artifact	Name
Policy register (all in-scope policies + owners)	`Policy Register – Purview – Governance`
Known-good policy baseline (drift reference)	`Policy Baseline – Purview – <date>`
Change record	`Change Record – Purview – ChangeControl`
Compliance Manager improvement tracker	`Improvement Actions – Compliance Manager – Cadence`
Alert triage queue	`Alert Triage – Purview – Ops`
Alert rollup report	`Alert Rollup – Purview – Report`
Role & access review record	`Access Review – Purview Roles – Quarterly`
KPI dashboard	`KPI Dashboard – Purview – ProgramHealth`
Operating review calendar	`Review Calendar – Purview – Operating`

Part A — Stand up the operating model (once)

Step 1 — Baseline the estate (Assess)

1. Confirm your least-privilege operations roles are in place (Requirement 2.2) and connect:

`Connect-IPPSSession`.

1. Run `PURVOPS-Discovery.ps1` to inventory every DLP, sensitivity-label, label-policy, auto-labeling, and

retention policy with its current **mode** (Enable / Test / Disable). It writes the `Policy Register – Purview – Governance`.

1. Review the register with stakeholders and record an **owner** for each policy family. Once the state is

agreed as known-good, run `PURVOPS-Discovery.ps1 -SaveBaseline` to write `Policy Baseline – Purview – <date>` — the reference all future drift checks diff against (Requirement 2.4).

Step 2 — Define the policy lifecycle + change control (Configure)

1. Adopt the lifecycle for every policy: **design** → **simulate/test** → **enforce** → **review.** New and changed

DLP and auto-labeling policies are created/edited in **simulation or test** mode first (`TestWithNotifications` / `TestWithoutNotifications`), never straight to enforce.

1. Stand up the change record `Change Record – Purview – ChangeControl` and the approval path + change

window (Requirement 2.9). Every live-policy edit becomes a change request → review → windowed change → recorded outcome → rollback plan.

1. Confirm the operational helper is safe by default: `PURVOPS-Remediation.ps1 -WhatIf` prints the intended

change and makes **none**; only `-Commit` writes, and it stamps the change record.

Step 3 — Wire alert triage + reporting (Configure)

1. Define the triage queue: categories (DLP / insider risk / information-protection / audit), severities,

and SLAs (Requirement 2.7). This is `Alert Triage – Purview – Ops`.

1. Confirm the Graph security read scope is consented (Requirement 2.3), then run `PURVOPS-Report.ps1` to

produce the first `Alert Rollup – Purview – Report` — alert volume by category, severity, and status.

1. Assign the **daily triage owner** and the **weekly rollup reviewer**.

Step 4 — Set up the Compliance Manager cadence (Configure)

1. In the portal, **Compliance Manager** → **Improvement actions**, review the actions, owners, and score.

Export the improvement actions to CSV (Compliance Manager has **no public API**, Requirement 2.5).

1. Run `PURVOPS-KpiReport.ps1 -ComplianceManagerCsv <export.csv>` to fold the improvement actions and

compliance score into `Improvement Actions – Compliance Manager – Cadence` and the KPI dashboard.

1. Assign each open improvement action an owner and due date; agree the **monthly** burn-down forum.

(Determination of which regulatory obligations apply remains with compliance/legal — this cadence tracks and evidences the technical actions, not the regulatory determination.)

Step 5 — Schedule the role & access review (Configure)

1. Capture the approved role membership as the attestation baseline (Requirement 2.6):

`PURVOPS-Validation.ps1 -SaveApprovedList` after the accountable owner confirms current membership is correct.

1. Schedule the **quarterly** review that will diff live membership against that approved list and produce

`Access Review – Purview Roles – Quarterly`.

Step 6 — Stand up the KPI dashboard + review calendar (Configure)

1. Agree the KPI set and thresholds (Requirement 2.8). Run `PURVOPS-KpiReport.ps1` to compose `KPI Dashboard

– Purview – ProgramHealth` from live telemetry (policy-mode distribution, alert volume, label/DLP coverage) plus the Compliance Manager export.

1. Publish `Review Calendar – Purview – Operating` with the daily/weekly/monthly/quarterly/annual cadences

and named owners, and schedule the recurring **program-health review**.

Part B — Run the program (recurring heartbeat)

RB-01 — Policy lifecycle & change control (weekly drift + quarterly review)

1. **Weekly:** run `PURVOPS-Discovery.ps1 -CompareBaseline` — it diffs the live estate against `Policy

Baseline – Purview – <date>` and reports **drift**: new, removed, or mode-changed policies. Investigate every drift item; an unexpected mode change (e.g. a policy silently moved test → enforce, or enforce → disabled) is a change-control event.

1. **To promote a piloted policy to enforce:** confirm its simulation/test results are clean, raise a change

request, then `PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "<name>" -WhatIf` to preview, and `-Commit` in the change window. **Pass:** mode is now `Enable` and the change is recorded. **Rollback:** `-Action RollbackPolicy -PolicyName "<name>" -Commit` returns it to test/disabled.

1. **Quarterly:** re-review every *live* policy against its business intent; retire or re-scope as needed

(each edit through change control). Re-baseline (`-SaveBaseline`) after the approved change set.

RB-02 — Compliance Manager improvement cadence (monthly)

1. Export improvement actions from Compliance Manager and run `PURVOPS-KpiReport.ps1 -ComplianceManagerCsv

<export.csv> `to refresh` Improvement Actions – Compliance Manager – Cadence`.

1. In the monthly forum, burn down actions: confirm completed ones, chase overdue ones, reassign as needed.

Record the compliance-score trend as a KPI. **Pass:** overdue actions trending down, score stable/up.

RB-03 — Alert triage & incident rollup (daily triage, weekly rollup)

1. **Daily:** the triage owner works `Alert Triage – Purview – Ops` — for each alert, classify

(true/false positive), action or escalate within its SLA, and note tuning needed. Insider-risk and communication-compliance alerts follow the **role-separated, privacy-by-design** handling; case **outcomes are deferred to HR/legal** — operations triages and routes, it does not adjudicate.

1. **Weekly:** run `PURVOPS-Report.ps1` to refresh `Alert Rollup – Purview – Report`. Review volume by

category/severity, mean triage time against SLA, and recurring false-positive sources to feed policy tuning (via RB-01 change control). **Pass:** high-severity alerts within SLA; false-positive rate falling.

RB-04 — Role & access review (quarterly)

1. Run `PURVOPS-Validation.ps1 -ApprovedListCsv <approved.csv>` to diff live Purview role-group membership

against the approved baseline (Requirement 2.6), producing `Access Review – Purview Roles – Quarterly`.

1. The accountable owner attests each membership: confirm, or remove departed/role-changed members under

change control. Flag any highly privileged account (e.g. multiple sensitive role groups) for justification. Re-save the approved list (`-SaveApprovedList`) after attestation. **Pass:** zero unexplained additions; all privileged memberships attested.

RB-05 — KPI dashboard & program-health review (monthly refresh, quarterly/annual review)

1. **Monthly:** run `PURVOPS-KpiReport.ps1` to refresh `KPI Dashboard – Purview – ProgramHealth`.

2. **Quarterly:** walk the dashboard in the program-health review — policies enforced vs test, alert volume

and triage time, improvement-action burn-down, compliance-score trend, privileged-role count — against thresholds. Decide tuning and scope changes; feed them back through RB-01.

1. **Annually:** reassess scope and coverage (new workloads, new regulations flagged by compliance) and set

next year's targets. **Pass:** KPIs within threshold or with an owned remediation plan.

Order of operations

`Baseline → define lifecycle + change control → wire triage/reporting → set CM cadence → schedule access review → stand up KPIs → then run the heartbeat: weekly drift · daily triage / weekly rollup · monthly CM + KPI · quarterly access + policy review · annual program review — every change -WhatIf-first and recorded.`

5. Testing & Validation Guide — Managing Microsoft Purview: Operations & Governance

Validate that the **operating model actually runs** — not that Purview is merely configured. Each test below has an explicit **pass/fail** signal and, where relevant, a **rollback**. Run the Part-A tests once when you stand the program up, then treat the Part-B tests as the acceptance gate for each recurring cycle. All checks are **read-only or** `-WhatIf` unless a step is explicitly a controlled change.

Test 1 — Policy inventory + drift detection (RB-01)

1. Run `PURVOPS-Discovery.ps1` and confirm it returns a non-empty inventory of DLP, label, auto-labeling,

and retention policies, each with a resolved **mode** (Enable / Test / Disable).

1. Run `PURVOPS-Discovery.ps1 -SaveBaseline`, then make **no** changes and run `-CompareBaseline`.

• **Pass:** the diff reports **zero drift**.

• **Fail:** drift reported with no change made → the baseline or comparison key is wrong; re-baseline.

1. In a **test** policy, change the mode (or add a benign test policy), re-run `-CompareBaseline`.

• **Pass:** the exact policy and mode change is reported as drift.

• **Rollback:** revert the test change; confirm the next diff is clean.

Test 2 — Config helper is safe by default (RB-01 / change control)

1. Run `PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "<a test policy>" -WhatIf`.

• **Pass:** it prints the intended change and confirms **no change was made**; re-running Discovery shows

the policy mode unchanged.

• **Fail:** any change occurs under `-WhatIf` → stop and do not use `-Commit` until fixed.

1. Only in a change window, against a **non-production/test** policy, run the same with `-Commit`.

• **Pass:** the mode changes as previewed **and** a row is written to `Change Record – Purview – ChangeControl`.

• **Rollback:** `-Action RollbackPolicy -PolicyName "<a test policy>" -Commit` returns it to its prior mode.

Test 3 — Alert rollup across categories (RB-03)

1. Run `PURVOPS-Report.ps1`. Confirm it authenticates to Graph and returns alerts grouped by **category**

(DLP / insider risk / information-protection / audit), **severity**, and **status**.

• **Pass:** counts render and totals reconcile with the Purview/Defender alert views for the same window.

• **Fail:** a permissions error → the Graph security read scope is not consented (Requirement 2.3).

1. Confirm the mean triage time (where status/timestamps are available) is computed against your SLA.

(If no alerts exist in the window, the script reports an empty rollup cleanly — this is a valid pass.)

Test 4 — Role & access review diff (RB-04)

1. Run `PURVOPS-Validation.ps1 -SaveApprovedList` to capture the current membership baseline.

2. Add a **test** account to a low-risk role group, then run `PURVOPS-Validation.ps1 -ApprovedListCsv <approved.csv>`.

• **Pass:** the test account is flagged as an **unapproved addition**.

• **Rollback:** remove the test account; the next run shows zero unexplained additions.

1. Confirm accounts in multiple sensitive role groups are flagged for justification.

Test 5 — Compliance Manager cadence + KPI dashboard (RB-02 / RB-05)

1. Export improvement actions from Compliance Manager, then run `PURVOPS-KpiReport.ps1 -ComplianceManagerCsv <export.csv>`.

• **Pass:** `Improvement Actions – Compliance Manager – Cadence` lists actions with owner/status/due date,

and `KPI Dashboard – Purview – ProgramHealth` renders the KPI set (policy-mode distribution, alert volume, coverage, improvement-action burn-down, compliance-score trend, privileged-role count).

• **Fail:** the CSV does not parse → re-export from the portal (column layout can change; the script reports the columns it found).

1. Confirm each KPI is compared to its agreed threshold and out-of-threshold values are highlighted.

Test 6 — The cadence actually fires (operating model)

1. Confirm `Review Calendar – Purview – Operating` has scheduled owners for daily triage, weekly drift +

rollup, monthly CM + KPI, quarterly access + policy review, and the annual program review.

• **Pass:** each cadence has a named owner and a scheduled occurrence; one full cycle of each has run and

produced its artifact.

• **Fail:** any cadence with no owner or no evidence of a run → assign and run before sign-off.

Acceptance gate (all must pass before declaring the program "operating")

• Discovery returns a full policy inventory with modes; baseline diff is clean, and a real change is

caught as drift.

• `PURVOPS-Remediation.ps1` makes no change under `-WhatIf`; `-Commit` changes as previewed **and** writes

a change record; rollback works.

• Alert rollup renders by category/severity/status and reconciles with the portal.

• Access-review diff flags an unapproved addition and privileged multi-role accounts.

• Compliance Manager export ingests; KPI dashboard renders all KPIs against thresholds.

• Every operating cadence has a named owner and one completed cycle on the calendar.

6. Troubleshooting Guide — Managing Microsoft Purview: Operations & Governance

Common failures when *operating* Purview, with the cause and the fix. Grouped by runbook. All fixes stay within the safe-by-default model — read-only checks first, changes only under `-WhatIf`-then-`-Commit`.

Connection & permissions

"The term 'Get-DlpCompliancePolicy' / 'Get-RoleGroup' is not recognized."

• *Cause:* not in a Security & Compliance PowerShell session. Those cmdlets only exist after

`Connect-IPPSSession`.

• *Fix:* `Import-Module ExchangeOnlineManagement; Connect-IPPSSession` with a modern-auth/MFA admin, then

re-run the script.

`PURVOPS-Report.ps1` **returns a permissions/authorization error from Graph.**

• *Cause:* the Graph **security** read scope is not consented.

• *Fix:* `Connect-MgGraph -Scopes 'SecurityAlert.Read.All'` (or `SecurityEvents.Read.All`); if admin consent

is required, an admin approves the scope once in Entra. Confirm with `Get-MgContext`.

A `Get-*` cmdlet errors with "not available" for a whole family (e.g. auto-labeling, insider risk).

• *Cause:* the capability is **not licensed/enabled** in your tenant, or your role lacks access. Purview is heavily plan-gated (E5 / E5 Compliance) and evolving.

• *Fix:* verify licensing and the role group (Requirement 2.2) in the portal; the discovery script logs a warning and continues with the families you *can* read — this is expected on lower plans.

RB-01 — Policy lifecycle & drift

`-CompareBaseline` **reports drift when nothing changed.**

• *Cause:* the baseline was saved against a different tenant/role scope, or a policy family became newly

visible/invisible due to a licensing/role change; the compare key is Family|Name.

• *Fix:* confirm you are connected as the same role that saved the baseline; if the estate is genuinely known-good, re-baseline with `-SaveBaseline`. Only re-baseline after an **approved** state.

Real drift appears but no one changed anything on purpose.

• *Cause:* an out-of-band edit — someone changed a policy mode outside change control, or an automated

process did.

• *Fix:* treat as a change-control incident. Cross-check the change record (`Change Record – Purview –

ChangeControl) and the unified audit log (Search-UnifiedAuditLog`) for who/when before re-baselining.

`PURVOPS-Remediation.ps1 -WhatIf` **seems to do nothing / you expected a change.**

• *Cause:* that is correct behavior — the tool is **safe by default**; without `-Commit` it previews only.

• *Fix:* review the previewed From→To mode, then re-run with `-Commit` **inside a change window** to apply.

`-Commit` **fails with a mode error on promote.**

• *Cause:* the policy is not in a promotable state, the target mode value is invalid for that policy type, or the simulation hasn't completed.

• *Fix:* confirm the policy's current mode with `PURVOPS-Discovery.ps1`; ensure auto-labeling simulation has

finished before promoting; valid rollback modes are `TestWithNotifications`, `TestWithoutNotifications`, `Disable`. Never force-promote a policy whose test results aren't clean.

RB-03 — Alert triage & rollup

The rollup is empty but you know alerts exist.

- *Cause:* the window (`-Days`) is too short, the alerts live on a surface not exposed via

`/security/alerts_v2` for your tenant, or they haven't synchronized.

- *Fix:* widen `-Days`; reconcile against the Purview/Defender portal for the same window; some Purview

alerts surface through the portal rather than the unified alerts API — note the gap and triage those in the portal.

Categories look wrong (everything lands in "Other").

- *Cause:* the best-effort categorizer keys off alert source/title text, which varies by tenant and evolves.
- *Fix:* adjust the category patterns in `PURVOPS-Report.ps1` to match your tenant's alert titles/sources;

the counts are still valid, only the bucketing needs tuning.

Mean time-to-resolve is blank.

- *Cause:* alerts have no `resolvedDateTime` (still open) or timestamps aren't populated.
- *Fix:* expected for open alerts; the KPI computes over resolved alerts only. Ensure your triage process

actually resolves/closes alerts so the metric has data.

RB-04 — Role & access review

The diff flags additions that are actually legitimate.

- *Cause:* the approved baseline is stale — legitimate changes happened but the baseline wasn't re-saved

after the last attestation.

- *Fix:* attest the additions with the accountable owner, then re-run `-SaveApprovedList` to refresh the

baseline. Only re-save **after** attestation.

`Get-RoleGroupMember` **returns nothing for a group that has members.**

- *Cause:* nested/queried membership or a role group managed outside the S&C surface, or a transient service

issue.

- *Fix:* confirm membership in the portal (**Roles & scopes** → **Permissions**); some role groups are managed in

the Purview portal directly — verify there and adjust the approved list accordingly.

RB-02 / RB-05 — Compliance Manager & KPIs

The Compliance Manager CSV won't parse / KPIs show "n/a".

- *Cause:* the export column layout differs from what the script expects (Compliance Manager has **no public

API**; the portal export format changes over time).

- *Fix:* the script prints the columns it found — re-export from **Compliance Manager → Improvement actions →

Export**, confirm a status and a due/target column exist, and (if renamed) point the detection at the right columns. This is a data-format issue, not a Purview fault.

Live policy-mode / role KPIs show "n/a (not connected)".

- *Cause:* `PURVOPS-KpiReport.ps1` was run without an S&C PowerShell session (those KPIs need it; CM/alert

KPIs are CSV-driven and independent).

- *Fix:* `Connect-IPPSSession` before running for the live KPIs; the CSV-driven KPIs still populate either way.

Compliance score dropped after a bulk "implemented" update.

- *Cause:* actions were marked complete without evidence, or a reassessment lowered credit.
- *Fix:* evidence each completion; do not game the score. Track the **trend**, and let compliance/legal own

which obligations apply — the cadence evidences technical actions, not the regulatory determination.

General

A scheduled read-only run stopped producing artifacts.

- *Cause:* an expired sign-in/token for the unattended context, or a module update changed a cmdlet.
- *Fix:* refresh the auth for the scheduled context; pin/verify module versions; keep **writes human-gated** —

only Discovery/Report/Validation/KPI runs should ever be scheduled, never `-Commit` changes.

Not sure whether a capability/path is available for your tenant.

- *Cause:* Purview is plan-gated and the portal evolves.
- *Fix:* verify in the M365 admin center and the Purview portal for **your** tenant before relying on a

feature; flag "verify your licensing/tenant" in your runbook where availability is uncertain — never assume.

7. Best Practices — Managing Microsoft Purview: Operations & Governance

Field-tested practices for running Purview as an operated, measured program. They assume the operating model in the Operating Guide is in place; the goal here is to keep it **effective, tuned, and defensible** over time.

Policy lifecycle & change control

- **Never edit a live enforcing policy without change control.** The classic Purview outage is a

well-intentioned off-hours edit to a live DLP or retention policy. Every change: request → review → windowed change → recorded outcome → rollback plan. Use `PURVOPS-Remediation.ps1 -WhatIf` first, always.

- **Simulation/test before enforce — no exceptions.** New and changed DLP and auto-labeling policies run in

`TestWithNotifications` / `TestWithoutNotifications` (auto-labeling simulation) until results are clean. Promotion to enforce is a reviewed, recorded event, not a default.

- **Re-baseline deliberately, not reflexively.** Only run `-SaveBaseline` after an *approved* change set, so

the baseline always represents a known-good, attested state — otherwise drift detection normalizes away the very changes you want to catch.

- **Treat unexpected drift as an incident.** A policy that moved test → enforce, or enforce → disabled,

without a matching change record is a control failure — investigate who/what/when before re-baselining.

- **Retire dead policies.** Disabled or superseded policies clutter the estate and confuse reviews; remove

them through change control rather than leaving them "just in case."

Alert triage & reporting

- **Own the queue or it owns you.** A named daily triage owner and a weekly rollup reviewer are the

difference between a working control and a dashboard of ignored red. SLAs by severity make triage measurable.

- **Tune relentlessly.** The weekly rollup exists to find recurring false-positive sources; feed each one

back into policy tuning (through RB-01 change control). A falling false-positive rate is a KPI, not a nicety.

- **Reconcile, don't trust blindly.** Periodically reconcile the scripted rollup against the Purview/Defender

portal views for the same window; alerting surfaces and APIs evolve, and silent gaps are the worst kind.

- **Insider risk & communication compliance: role-separate and privacy-by-design.** Keep *administrators*

(who configure policy) separate from *investigators/reviewers* (who see case content). Use pseudonymization where the feature supports it. Operations **triages and routes**; **case outcomes are HR/legal determinations** — never adjudicated by the ops function.

Compliance Manager cadence

- **Improvement actions need owners and due dates, or they rot.** The monthly burn-down is where the

compliance score is actually earned. Chase overdue actions; reassign when owners change.

• **Score is a trend, not a target to game.** Track the trajectory. A one-time spike from bulk-marking

actions "implemented" without evidence is a finding waiting to happen — evidence each completion.

• **Remember the API gap.** Compliance Manager has **no supported public API**; the export is manual by

design. Build the monthly export into the calendar so the KPI never goes stale silently.

• **Regulatory applicability is compliance's call.** The cadence evidences *technical* actions; *which*

obligations apply, and the records/retention schedule behind them, remain with compliance, records, and legal.

Role & access reviews

• **Quarterly, against an approved baseline, attested by the accountable owner.** A list of members is not a

review; the diff against approved membership plus an attestation is.

• **Least privilege is a living state.** People change roles and projects end — memberships must follow.

Flag and justify every account holding multiple sensitive role groups.

• **Preserve separation of duties.** Do not let the access-review process quietly re-merge administrator and

investigator duties for HR/privacy-sensitive capabilities.

KPIs & program health

• **Measure what proves the program works, not what's easy to count.** Good defaults: % policies enforced vs

test, alert volume + mean triage time vs SLA, improvement-action burn-down, compliance-score trend, privileged-role count, policy-drift incidents. Each with a threshold and an owner.

• **A KPI without a forum is decoration.** The quarterly program-health review must *act* on the dashboard —

decide tuning, scope, and remediation, and record the decisions.

• **Reassess scope annually.** New workloads, new regulations (flagged by compliance), and new Purview

capabilities change what "healthy coverage" means — set next year's targets deliberately.

Automation & safety

• **Read-only or `-WhatIf` by default; `-Commit` is the exception.** All five scripts lean read-only; the one

that writes gates every change behind an explicit flag and a change record.

• **No secrets in code — ever.** Scripts use interactive modern-auth (`Connect-IPPSSession` /

`Connect-MgGraph`) or a managed identity; never hardcode credentials, tokens, or app secrets.

• **Verify licensing and portal paths for *your* tenant.** Purview is heavily plan-gated and evolving — confirm each capability and path before relying on it, and flag "verify your licensing/tenant" in your own runbooks where availability is uncertain.

• **Schedule the read-only chain; keep writes human-gated.** Discovery, reporting, validation, and KPI runs can be scheduled; policy promotions and role removals stay behind change control and human approval.

Operating discipline (the short version)

> *Pilot before you enforce. Record before you change. Triage before it triages you. Attest before you assume. Measure before you claim. Re-baseline only what's approved.*

8. Deliverables Summary — Managing Microsoft Purview: Operations & Governance

Everything included in this solution and exactly how each piece is used to **run Microsoft Purview as an operated, measured program**.

Documentation

Document	What it gives you
Solution Overview	The Purview operating model — policy lifecycle, Compliance Manager cadence, alert triage + reporting, role/access reviews, change control, and KPI dashboards — and where governance determinations stay with compliance/legal.
Requirements Guide	The nine operations-specific prerequisites (implemented foundation, role-separated ops roles, toolchain + Graph security scopes, policy baseline, Compliance Manager export, approved role list, alert-triage SLAs, KPI set, change control) — each with Verify / Implement-if-missing — over the six category requirements.
Operating Guide	Part A to stand up the operating model once; Part B the recurring heartbeat as five named runbooks (RB-01…RB-05) with pass/fail and rollback.
Testing & Validation Guide	Six tests with explicit pass/fail + rollback and an acceptance gate proving the model actually runs (drift caught, `-WhatIf` safe, alerts roll up, access diff flags, KPIs render, cadences fire).

Document	What it gives you
Troubleshooting Guide	The common operating failures — connection/scope errors, drift false alarms, empty rollups, Compliance Manager export parsing, role-diff mismatches — with causes and fixes.
Best Practices	Durable practices for lifecycle/change control, triage/tuning, the CM cadence, access reviews, KPIs, and automation safety.
Executive Summary	A one-page brief for leadership on operating Purview as a measured program and why it de-risks compliance over time.

PowerShell tools (5) — Security & Compliance PowerShell / Microsoft Graph

All are **read-only or `-WhatIf` by default**; the one that writes gates every change behind `-Commit` and a change record. None contain credentials or secrets — they use interactive modern auth.

Script	Role	What it does	Default
PURVOPS-Discovery.ps1	Discovery / drift	Inventories every DLP, sensitivity-label, label-policy, auto-labeling, and retention policy with its mode; saves a baseline and reports **drift** against it.	Read-only
PURVOPS-Report.ps1	Reporting	Rolls up triggered alerts from Microsoft Graph (`/security/alerts_v2`) across DLP / insider-risk / information-protection / audit by category, severity, status, and triage time.	Read-only
PURVOPS-Validation.ps1	Validation	Enumerates Purview role groups and members and diffs them against an approved-membership baseline for the quarterly access review.	Read-only

Script	Role	What it does	Default
PURVOPS-KpiReport.ps1	Reporting / KPIs	Composes the program-health KPI dashboard from live telemetry plus the Compliance Manager improvement-action export (no public API — CSV in).	Read-only
PURVOPS-Remediation.ps1	Config / remediation	Operational config helper — promotes a piloted policy to enforce or rolls it back — `-WhatIf` first, `-Commit` writes and stamps the change record.	`-WhatIf` (safe)

Named operating artifacts (created and reused verbatim)

`Policy Register – Purview – Governance` · `Policy Baseline – Purview – <date>` · `Change Record – Purview – ChangeControl` · `Improvement Actions – Compliance Manager – Cadence` · `Alert Triage – Purview – Ops` · `Alert Rollup – Purview – Report` · `Access Review – Purview Roles – Quarterly` · `KPI Dashboard – Purview – ProgramHealth` · `Review Calendar – Purview – Operating`.

Operating cadences

Daily alert triage · weekly policy-drift + alert rollup · monthly Compliance Manager burn-down + KPI refresh · quarterly role/access review + live-policy review · annual program-health review and scope reassessment.

Outcome

Microsoft Purview run as a **living program** — policies piloted then enforced and reviewed on a cadence, alerts triaged and tuned, improvement actions burned down, roles attested, and KPIs proving health — so the program stays **effective, tuned, and defensible** year over year. Regulatory, records, and legal determinations remain with the accountable governance functions; this solution supplies the technical operating method, the runbooks, and the evidence.

Boundaries (what this solution does not decide)

- **Retention schedules, records file plans, and legal holds** — determined by records/legal; this operates

and evidences the technical policies.

- **Insider-risk / communication-compliance case outcomes** — HR/legal determinations; operations triages

and routes under role separation and privacy-by-design.

• **Regulatory applicability and the compliance score's obligations** — compliance/legal own the determination; the cadence tracks and evidences the technical actions.

• **Capability availability** — Purview is heavily licensing-gated and evolving; verify each feature and portal path for your own tenant.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PURVOPS-Discovery.ps1

Purpose: Builds the authoritative policy register for the operating model: every DLP, sensitivity-label, label-policy, auto-labeling, and retention policy with its current MODE (Enable / Test / Disable). Optionally saves a known-good baseline (-SaveBaseline) and, on later runs, compares live state to that baseline (-CompareBaseline) to report DRIFT — new, removed, or mode-changed policies since the last review. Read-only; uses Security & Compliance PowerShell. HOW TO RUN Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) .\PURVOPS-Discovery.ps1 # inventory only, to console + optional CSV .\PURVOPS-Discovery.ps1 -SaveBaseline -BaselinePath .\Policy-Baseline.json .\PURVOPS-Discovery.ps1 -CompareBaseline -BaselinePath .\Policy-Baseline.json PowerShell 5.1+/7+; requires the ExchangeOnlineManagement module (Connect-IPPSSession). EXPECTED RESULTS A per-policy table (Family, Name, Mode, Enabled) and, with -CompareBaseline, a DRIFT report listing Added / Removed / ModeChanged policies. Zero drift against an unchanged tenant = pass. .NOTES Purview — Operations & Governance. Read-only. Verify capability availability/licensing for your tenant; cmdlet availability depends on the compliance plan (E5 / E5 Compliance for advanced DLP/auto-labeling). > [CmdletBinding()] param([string]$CsvPath, [string]$BaselinePath = '.\Policy-Baseline.json', [switch]$SaveBaseline, [switch]$CompareBaseline)

How to run: Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) .\PURVOPS-Discovery.ps1 # inventory only, to console + optional CSV .\PURVOPS-Discovery.ps1 -SaveBaseline -BaselinePath .\Policy-Baseline.json .\PURVOPS-Discovery.ps1 -CompareBaseline -BaselinePath .\Policy-Baseline.json PowerShell 5.1+/7+; requires the ExchangeOnlineManagement module (Connect-IPPSSession). EXPECTED RESULTS A per-policy table (Family, Name, Mode, Enabled) and, with -CompareBaseline, a DRIFT report listing Added / Removed / ModeChanged policies. Zero drift against an unchanged tenant = pass. .NOTES Purview — Operations & Governance. Read-only. Verify capability availability/licensing for your tenant; cmdlet availability depends on the compliance plan (E5 / E5 Compliance for advanced DLP/auto-labeling). > [CmdletBinding()] param([string]$CsvPath, [string]$BaselinePath = '.\Policy-Baseline.json', [switch]$SaveBaseline, [switch]$CompareBaseline)

PURVOPS-KpiReport.ps1

Purpose: Composes "KPI Dashboard - Purview - ProgramHealth": policy-mode distribution (enforced vs test), privileged-role count, and — from the Compliance Manager improvement-actions CSV (Compliance Manager has NO supported public API, so the CSV is exported from the portal) — improvement-action burn-down and compliance-score inputs. Optionally folds in a prior alert-rollup CSV for alert-volume/triage KPIs. Read-only; each KPI is compared to a threshold you pass. HOW TO RUN Connect-IPPSSession # for the live policy-mode + role KPIs (no secrets) Export improvement actions: Purview portal > Compliance Manager > Improvement actions > Export .\PURVOPS-KpiReport.ps1 -ComplianceManagerCsv .\CM-Improvement-Actions.csv ` -AlertRollupCsv .\Alert-Rollup.csv -CsvPath .\KPI-Dashboard.csv PowerShell 5.1+/7+; live KPIs need Security & Compliance PowerShell; CM/alert KPIs are CSV-driven. EXPECTED RESULTS A KPI table (name, value, threshold, status) covering policy-mode %, privileged-role count, improvement-action burn-down, compliance-score, and (if supplied) alert volume. Out-of-threshold KPIs are flagged. Missing inputs degrade gracefully (that KPI shows "n/a"). .NOTES Purview — Operations & Governance. Read-only. Compliance Manager has no public API — the

improvement export is manual by design; column layout can change, so the script reports the columns it found. Regulatory applicability behind the score is a compliance/legal determination, not this report's. > [CmdletBinding()] param([string]$ComplianceManagerCsv, [string]$AlertRollupCsv, [string]$CsvPath, [double]$EnforcedPctThreshold = 80, # KPI: >= this % of DLP/auto-labeling policies enforcing [int]$MaxOpenOverdueActions = 0, # KPI: overdue improvement actions should trend to this [int]$MaxPrivilegedAccounts = 10 # KPI: accounts in sensitive role groups)

How to run: Connect-IPPSSession # for the live policy-mode + role KPIs (no secrets) Export improvement actions: Purview portal > Compliance Manager > Improvement actions > Export .\PURVOPS-KpiReport.ps1 -ComplianceManagerCsv .\CM-Improvement-Actions.csv ` -AlertRollupCsv .\Alert-Rollup.csv -CsvPath .\KPI-Dashboard.csv PowerShell 5.1+/7+; live KPIs need Security & Compliance PowerShell; CM/alert KPIs are CSV-driven. EXPECTED RESULTS A KPI table (name, value, threshold, status) covering policy-mode %, privileged-role count, improvement-action burn-down, compliance-score, and (if supplied) alert volume. Out-of-threshold KPIs are flagged. Missing inputs degrade gracefully (that KPI shows "n/a"). .NOTES Purview — Operations & Governance. Read-only. Compliance Manager has no public API — the improvement export is manual by design; column layout can change, so the script reports the columns it found. Regulatory applicability behind the score is a compliance/legal determination, not this report's. > [CmdletBinding()] param([string]$ComplianceManagerCsv, [string]$AlertRollupCsv, [string]$CsvPath, [double]$EnforcedPctThreshold = 80, # KPI: >= this % of DLP/auto-labeling policies enforcing [int]$MaxOpenOverdueActions = 0, # KPI: overdue improvement actions should trend to this [int]$MaxPrivilegedAccounts = 10 # KPI: accounts in sensitive role groups)

PURVOPS-Remediation.ps1

Purpose: The one write-capable operating tool, safe by default. Performs the routine lifecycle transitions under change control: PromotePolicy (move a DLP or auto-labeling policy from test/simulation to enforce, Mode=Enable) and RollbackPolicy (return it to a test/disabled mode). With no -Commit it PREVIEWS the change and makes NONE. On -Commit it applies the change and appends a row to the change record CSV (who/when/what/from/to). No credentials or secrets in code. HOW TO RUN Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) Preview (default, no change): .\PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "Finance DLP - Pilot" -WhatIf Apply inside a change window: .\PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "Finance DLP - Pilot" -Commit .\PURVOPS-Remediation.ps1 -Action RollbackPolicy -PolicyName "Finance DLP - Pilot" -RollbackMode TestWithNotifications -Commit PowerShell 5.1+/7+; requires Security & Compliance PowerShell (Set-DlpCompliancePolicy / Set-AutoSensitivityLabelPolicy). EXPECTED RESULTS -WhatIf: prints the intended From->To mode and confirms NO change was made. -Commit: mode changes as previewed AND a row is written to the change record CSV. .NOTES Purview — Operations & Governance. HIGH-IMPACT: only promote a policy whose simulation/test results are clean, only in a change window, with approval. Retention/records changes and any legal-hold action are DEFERRED to records/legal and are intentionally NOT automated here. Verify licensing/tenant. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][ValidateSet('PromotePolicy','RollbackPolicy')][string]$Action, [Parameter(Mandatory=$true)][string]$PolicyName, [ValidateSet('DLP','AutoLabeling')][string]$PolicyType = 'DLP', [ValidateSet('TestWithNotifications','TestWithoutNotifications','Disable')][string]$RollbackMode = 'TestWithNotifications', [switch]$Commit, [string]$ChangeRecordCsv = '.\Change-Record-Purview.csv')

How to run: Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) Preview (default, no change): .\PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "Finance DLP - Pilot" -WhatIf

Apply inside a change window: .\PURVOPS-Remediation.ps1 -Action PromotePolicy -PolicyName "Finance DLP - Pilot" -Commit .\PURVOPS-Remediation.ps1 -Action RollbackPolicy -PolicyName "Finance DLP - Pilot" -RollbackMode TestWithNotifications -Commit PowerShell 5.1+/7+; requires Security & Compliance PowerShell (Set-DlpCompliancePolicy / Set-AutoSensitivityLabelPolicy). EXPECTED RESULTS -WhatIf: prints the intended From->To mode and confirms NO change was made. -Commit: mode changes as previewed AND a row is written to the change record CSV. .NOTES Purview — Operations & Governance. HIGH-IMPACT: only promote a policy whose simulation/test results are clean, only in a change window, with approval. Retention/records changes and any legal-hold action are DEFERRED to records/legal and are intentionally NOT automated here. Verify licensing/tenant. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory=$true)][ValidateSet('PromotePolicy','RollbackPolicy')][string]$Action, [Parameter(Mandatory=$true)][string]$PolicyName, [ValidateSet('DLP','AutoLabeling')][string]$PolicyType = 'DLP', [ValidateSet('TestWithNotifications','TestWithoutNotifications','Disable')][string]$RollbackMode = 'TestWithNotifications', [switch]$Commit, [string]$ChangeRecordCsv = '.\Change-Record-Purview.csv')

PURVOPS-Report.ps1

Purpose: Produces the periodic "Alert Rollup - Purview - Report": triggered alerts pulled from Microsoft Graph (/security/alerts_v2), grouped by CATEGORY (DLP / insider-risk / information-protection / audit-other), SEVERITY, and STATUS, with mean time-to-resolve where timestamps allow — the input to the weekly triage review and the alert-volume/triage-time KPIs. Read-only. HOW TO RUN Connect-MgGraph -Scopes 'SecurityAlert.Read.All' # or SecurityEvents.Read.All; no secrets in code .\PURVOPS-Report.ps1 -Days 7 .\PURVOPS-Report.ps1 -Days 30 -CsvPath .\Alert-Rollup.csv PowerShell 5.1+/7+; requires Microsoft.Graph PowerShell. Alert surfaces evolve — reconcile the totals against the Purview/Defender portal for the same window. EXPECTED RESULTS A rollup table by category/severity/status plus counts and mean resolve time. An empty window (no alerts) reports cleanly and is a valid pass. .NOTES Purview — Operations & Governance. Read-only. Insider-risk/communication-compliance alert CONTENT is role-separated and privacy-sensitive; this rollup reports COUNTS/metadata for operations, not case outcomes (deferred to HR/legal). Verify licensing/tenant for the alerting surfaces in scope. > [CmdletBinding()] param([int]$Days = 7, [string]$CsvPath)

How to run: Connect-MgGraph -Scopes 'SecurityAlert.Read.All' # or SecurityEvents.Read.All; no secrets in code .\PURVOPS-Report.ps1 -Days 7 .\PURVOPS-Report.ps1 -Days 30 -CsvPath .\Alert-Rollup.csv PowerShell 5.1+/7+; requires Microsoft.Graph PowerShell. Alert surfaces evolve — reconcile the totals against the Purview/Defender portal for the same window. EXPECTED RESULTS A rollup table by category/severity/status plus counts and mean resolve time. An empty window (no alerts) reports cleanly and is a valid pass. .NOTES Purview — Operations & Governance. Read-only. Insider-risk/communication-compliance alert CONTENT is role-separated and privacy-sensitive; this rollup reports COUNTS/metadata for operations, not case outcomes (deferred to HR/legal). Verify licensing/tenant for the alerting surfaces in scope. > [CmdletBinding()] param([int]$Days = 7, [string]$CsvPath)

PURVOPS-Validation.ps1

Purpose: Drives the quarterly access review. Enumerates Purview/compliance role groups and their members and either saves the current membership as the approved baseline (-SaveApprovedList) or compares live membership to an approved CSV (-ApprovedListCsv), flagging unapproved ADDITIONS, missing (removed) members, and accounts holding MULTIPLE sensitive role groups (for justification).

Read-only. HOW TO RUN Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) .\PURVOPS-Validation.ps1 -SaveApprovedList -ApprovedListCsv .\Approved-Roles.csv .\PURVOPS-Validation.ps1 -ApprovedListCsv .\Approved-Roles.csv PowerShell 5.1+/7+; requires Security & Compliance PowerShell (Get-RoleGroup / Get-RoleGroupMember). EXPECTED RESULTS A membership table by role group, and (with an approved CSV) an Additions/Removals diff plus a multi-role privileged-account list. Zero unexplained additions + all privileged accounts attested = pass. .NOTES Purview — Operations & Governance. Read-only. This reports membership for attestation; the ACCOUNTABLE OWNER attests and any removal is done under change control. Preserve administrator/investigator role separation for insider-risk / communication-compliance. Verify role-group availability for your tenant. > [CmdletBinding()] param([string]$ApprovedListCsv = '.\Approved-Roles.csv', [switch]$SaveApprovedList, Role groups treated as "sensitive" for the multi-role privilege flag (adjust to your governance). [string[]]$SensitiveRoleGroups = @('ComplianceAdministrator','Compliance Administrator','Compliance Data Administrator', 'Insider Risk Management','InsiderRiskManagement','eDiscovery Manager','eDiscovery Administrator', 'Information Protection','Information Protection Admins','Records Management','Audit Manager'))

How to run: Connect-IPPSSession # modern-auth admin sign-in (no secrets in code) .\PURVOPS-Validation.ps1 -SaveApprovedList -ApprovedListCsv .\Approved-Roles.csv .\PURVOPS-Validation.ps1 -ApprovedListCsv .\Approved-Roles.csv PowerShell 5.1+/7+; requires Security & Compliance PowerShell (Get-RoleGroup / Get-RoleGroupMember). EXPECTED RESULTS A membership table by role group, and (with an approved CSV) an Additions/Removals diff plus a multi-role privileged-account list. Zero unexplained additions + all privileged accounts attested = pass. .NOTES Purview — Operations & Governance. Read-only. This reports membership for attestation; the ACCOUNTABLE OWNER attests and any removal is done under change control. Preserve administrator/investigator role separation for insider-risk / communication-compliance. Verify role-group availability for your tenant. > [CmdletBinding()] param([string]$ApprovedListCsv = '.\Approved-Roles.csv', [switch]$SaveApprovedList, Role groups treated as "sensitive" for the multi-role privilege flag (adjust to your governance). [string[]]$SensitiveRoleGroups = @('ComplianceAdministrator','Compliance Administrator','Compliance Data Administrator', 'Insider Risk Management','InsiderRiskManagement','eDiscovery Manager','eDiscovery Administrator', 'Information Protection','Information Protection Admins','Records Management','Audit Manager'))